Agentic AI for Scientific Computing

Hackathon intro

Nat Trask

ENM5320, Spring 2026

April 27, 2026

Today

1. Build a ReAct agent from scratch (about 20 lines of Python, no framework).
2. Turn it loose on **adaptive mesh refinement** for a 2D Poisson problem on an L-shape.
3. Turn it loose on **black-box optimization** of a projectile simulator.

Free LLM backend: Google Gemini 2.0 Flash. Get a key at `aistudio.google.com/apikey` (Google account, no credit card).

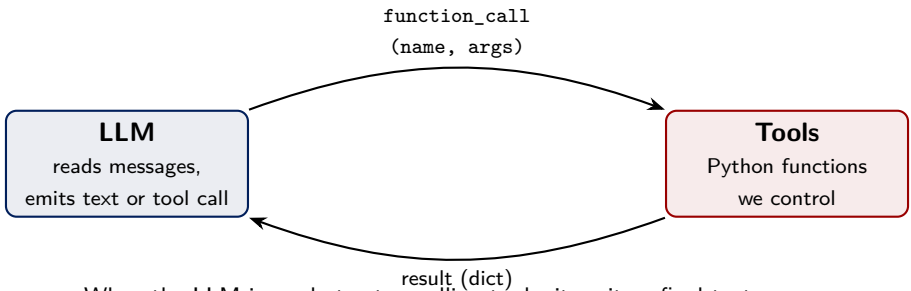
What is an LLM-based agent?

Agent = LLM + tools + loop.

- A bare LLM takes text in, emits text out. One shot.
- An *agent* is an LLM running in a loop, given access to **tools**: Python functions it can request we run on its behalf.
- At each turn the LLM reads the running conversation, decides to either:
  - emit a final text answer, or
  - call a tool with structured arguments.
- When it calls a tool, we run the function and feed the result back as the next turn.

Called *ReAct* (Reason + Act) in the literature, Yao et al. 2022.

The loop, as a picture



When the LLM is ready to stop calling tools, it emits a final text answer and the loop exits.

What a tool is

Two pieces, written by us: an ordinary Python function, and a JSON-schema description of its signature that we hand to the model.

Python function (we call it)

```python
def calculator(op, a, b):
    ops = {"add": a+b, "sub": a-b,
           "mul": a*b, "div": a/b}
    return {"result": ops[op]}
```

JSON schema (the model reads it)

```
{
  "name": "calculator",
  "description": "One arithmetic op.",
  "parameters": {
    "type": "object",
    "properties": {
      "op": {"type": "string",
             "enum": ["add","sub","mul","div"
                      ]},
      "a":  {"type": "number"},
      "b":  {"type": "number"},
    },
    "required": ["op","a","b"],
  },
}
```

What each schema field controls:

- `name`, `description` — whether the model picks this tool over others.

Tool calling: the round trip

Each turn the model emits either plain text (final answer) or a structured tool call.

1. Model's reply contains a `function_call` part:

```
response.candidates[0].content.parts[0].function_call
  .name = "calculator"
  .args = {"op": "mul", "a": 13, "b": 47}
```

2. We dispatch by name and run the Python function:

```
result = tool_fns[name](**args)    # = {"result": 611}
```

3. We send the result back as a `function_response` on the next turn:

```
Part.from_function_response(name="calculator",
                            response={"result": 611})
```

Loop: the model sees its own previous call, the result, and decides whether to call another tool or stop and return text.

The ReAct loop in pseudocode

```
def run_agent(system_prompt, user_prompt, tool_schemas, tool_fns,
              max_steps=12):
    messages = [user_prompt]
    for step in range(max_steps):
        response = llm(system_prompt, messages, tools=tool_schemas)
        messages.append(response)
        if response.is_tool_call:
            for call in response.tool_calls:
                try:
                    result = tool_fns[call.name](**call.args)
                except Exception as e:
                    result = {"error": str(e)}
                messages.append(result)
        else:
            return response.text  # final answer
```

The notebook has this, fleshed out, in about 30 lines.

Two slots: `system_prompt` vs `user_prompt`

`run_agent` takes two prompt strings and puts them in different slots:

- **`system_prompt`** — the *rules of the game*. Identity, methodology, what the tools are for, budgets, how to stop. Sent on every turn as a persistent header; never gets drowned out as observations pile up.
- **`user_prompt`** — the *specific ask* that triggers this run. Becomes the first message of the conversation; tool calls and observations append after it.

For example, the B1 optimization agent splits them like this:

```
B1_SYSTEM = """You optimize a black-box function (angle in degrees,
in (0, 90)) returning a range in meters. Budget: at most 12 evaluate()
calls. Use history if helpful, then call propose_answer with your guess."""

B1_USER = "Find the optimal angle. Budget: 12 evaluations."
```

Rule of thumb: durable rules about *how to behave* go in `system_prompt`; the specific *thing to do this run* goes in `user_prompt`.

Errors are observations

Tool calls will sometimes fail: the model invents a nonexistent argument, asks for a mesh ID that doesn't exist, passes a string instead of a number.

The loop wraps every dispatch in `try/except` and feeds the exception back as the tool's observation:

```
try:
    result = tool_fns[name](**args)
except Exception as e:
    result = {"error": f"{type(e).__name__}: {e}"}
```

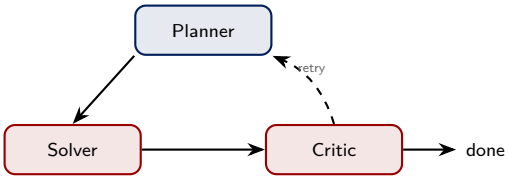
The model reads the error on the next turn and, with a reasonable prompt, retries with correct arguments.

In the tutorial you will see this trigger in practice. Do not treat it as a bug.

ReAct is the simplest option. Two others.

Multi-agent (LangGraph, AutoGen, CrewAI)

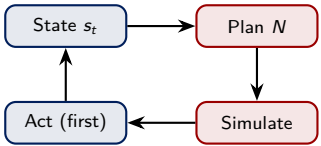
A state machine of specialized agents. Each node is an LLM with its own system prompt and tool subset; edges encode control flow.



Beats ReAct when roles are genuinely distinct, branches run in parallel, or each role needs its own guardrails.

MPC-style (Model Predictive Control)

At each step: look ahead, plan the next N actions against a world model, execute only the first action, then re-plan.
Tree-of-Thoughts is a related variant that searches branches.



Beats ReAct when the cost of a wrong action is high and you have a cheap, trusted forward simulator or world model.

Task 1: adaptive mesh refinement on an L-shape

Poisson with uniform forcing:

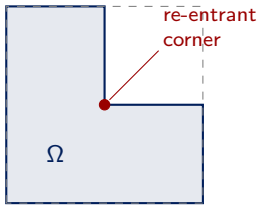
$$-\Delta u = 1 \text{ in } \Omega, \qquad u = 0 \text{ on } \partial\Omega.$$

The re-entrant corner at the origin gives a singular solution

$$u \sim r^{2/3} \sin(2\theta/3)$$

so $u \in H^{5/3-\epsilon}$, $u \notin H^2$.

Task for the agent: drive the H^1 error below a tolerance by choosing where to refine.



Expected convergence

P1 finite elements:

- **Uniform refinement.** Asymptotic H^1-seminorm rate is $O(h^{2/3})$. Pre-asymptotically you will often see rates of 0.7 to 0.95.
- **Adaptive refinement** (driven by a local error indicator). Optimal rate $O(N^{-1/2})$, i.e. rate 1 against $h \sim N^{-1/2}$.

The gap between these two rates is the entire point of adaptive meshing on this problem. We will make the agent rediscover it.

What "adaptive refinement" actually means

1. A local error indicator η_K. For each element K, a computable proxy for how much error lives there. We use the *gradient-jump indicator*:

$$\eta_K^2 \;=\; \tfrac{1}{2} \sum_{e \,\in\, \partial K \cap \Omega^\circ} h_e \left\| [\![\nabla u_h \cdot n_e]\!] \right\|^2 .$$

P1 elements have discontinuous gradients across element edges, and the size of the jump correlates with the local error.

2. Dörfler marking. Pick a bulk fraction $\theta \in (0, 1)$. Mark the smallest set of elements $\mathcal{M}$ whose squared indicators capture a fraction θ of the total:

$$\sum_{K \in \mathcal{M}} \eta_K^2 \;\geq\; \theta \sum_{K} \eta_K^2 .$$

Refine those; leave the rest. In practice: sort elements by η_K descending, walk down the list until the running sum hits $\theta \cdot$ total, refine everything you walked past.

$\theta = 0.5$ is the textbook default. Smaller θ = focus on the worst offenders (slower mesh growth). Larger θ = closer to uniform refinement.

Tools the FEM agent gets

Each tool takes a `mesh_id` (integer) and returns a dict.

- `solve_poisson(mesh_id)` $\rightarrow$ {dofs, H^1 error}
- `local_indicators(mesh_id)` $\rightarrow$ per-element gradient-jump indicators (summary stats)
- `uniform_refine(mesh_id)` $\rightarrow$ new mesh id
- `refine_by_threshold(mesh_id, theta)` $\rightarrow$ Dorfler-mark and refine
- `check_convergence_rate()` $\rightarrow$ log-log slope of last 3 solves vs theory
- `stop(reason)`

Guardrails built in: `MAX_DOFS = 20000`; every tool is `try/except`-wrapped.

Three prompts you will write

A1. Uniform refinement agent. Only allowed to use `uniform_refine`.

A2. Adaptive refinement agent. Uses `refine_by_threshold` with Dorfler marking ($\theta = 0.5$).

A3. Rate-checking agent. Calls `check_convergence_rate` every cycle and adapts θ if the empirical rate drops.

Plot at the end: H^1 error vs $h \sim 1/\sqrt{N}$ on log-log, all three agents overlaid, with theoretical slopes $2/3$ and 1.

Task 2: projectile with drag

Quadratic drag:

$$\ddot{x} = -c_d |v| \, \dot{x}, \qquad \ddot{y} = -g - c_d |v| \, \dot{y}.$$

Parameters: $v_0 = 50$ m/s, $c_d = 0.01$ m^{-1}, $g = 9.81$ m/s^2.

Task for the agent: find the launch angle $\theta \in (0^\circ, 90^\circ)$ that maximizes horizontal range.

Without drag the optimum is 45°; with drag it shifts lower. The agent does not see the ODE; it only sees the outputs of a black-box `evaluate(angle)` tool.

Tools the optimization agent gets

- `evaluate(angle_deg)` $\rightarrow$ horizontal range (m)
- `history()` $\rightarrow$ all past (angle, range) evaluations
- `propose_answer(angle_deg, rationale)` $\rightarrow$ final guess
- (bonus) `evaluate_height(angle_deg)` $\rightarrow$ max altitude

Three prompts you will write:

- **B1.** Trial agent. Small budget, any search strategy.
- **B2.** Finite-difference gradient agent. Centered FD estimate of $dR/d\theta$, step in the ascent direction. Callback to Lecture 3.
- **B3** (bonus). Maximize range subject to $\max_t y(t) \leq 30$ m.

The verifier-agent pattern

After each of the two main tasks, we run a **second, separate agent** that does not trust the first:

- **Part 2 verifier.** Designs its own manufactured-solution test. Picks a smooth u_{exact}, computes $f = -\Delta u_{\text{exact}}$, solves with $u_h = u_{\text{exact}}$ on $\partial\Omega$, and checks the errors converge at the expected rates.
- **Part 3 verifier.** Takes the claimed optimal angle and re-integrates the ODE with `solve_ivp(method="RK45", rtol=1e-10)`. Reports the discrepancy against `odeint`.

A verifier is just another agent with a different system prompt and a different tool set.

Before we start

1. Get a free Gemini API key at `aistudio.google.com/apikey`. Sign in with any Google account.
2. Open the Colab notebook (linked on the course page, Apr 27).
3. In Colab, click the *Secrets* panel (key icon, left sidebar) and add a secret named `GEMINI_API_KEY` with your key as the value.
4. Run the Part 0 cells to confirm the client connects.

If you already have an Anthropic, OpenAI, or Groq key you can swap backends by editing a few lines in the `run_agent` function (noted in the notebook).

Let's go.

Open the notebook and run Part 0.